스마트 환경제어 홈IoT

MQTT 기반 분산 제어 시스템

센서 · 중앙 제어기 · 액추에이터 · 웹 콘솔을

MQTT로 연결한 분산 IoT 시스템

3조 정귀인 · 황지용 · 김도경

프로젝트 배경 및 목표

프로젝트 배경

기술적 통합 시도

학습한 센서 제어, 액추에이터 구동, MQTT 통신, 웹 연동 기술을 실제 시스템에 적용하는 종합 프로젝트

학습 내용의 심화

이론적으로 학습한 내용을 프로젝트 형태의 결과물로 완성하며 기술적 이해도와 응용력 향상

주요 목표

협업 경험 확보

팀원별 역할 분담과 기능 통합 과정을 통한 실무적인 협업 프로세스 경험

완성된 시스템 구현

단순한 기능 구현을 넘어, 상호 유기적으로 동작하는 하나의 스마트 홈IoT 시스템 완성

프로젝트 개요

node_a
환경 센서

조도·온도·습도 측정

→ MQTT 발행

node_c
중앙 제어기

센서값 수신

→ 임계값 판단

→ 명령 발행

node_b
액추에이터

조명(WS2812)

창문(서보) 제어

web_console
관제 UI

상태 확인

수동 제어

로그 표시

시스템 제어 흐름

node_a
(환경 감지)

→ env

node_c
(중앙 판단)

→ cmd

node_b
(장치 제어)

status / heartbeat (to web_console)

전체 시스템 구성

사용자 계층 (User Layer)

web_console
Flask 웹 서버, 상태 모니터링/수동 제어

MQTT

중앙 관리 계층 (Control Layer)

MQTT Broker
중앙 메시지 중계 허브

node_c
Pico 2 W, 임계값 판단, 명령 발행, 상태 관리

MQTT

현장 계층 (Field Layer)

node_a
Pico 2 W, CDS 조도 센서, DHT 온습도 센서

node_b
Pico 2 W, WS2812 RGB 조명, SG90 서보(창문)

node_a
환경 데이터 수집 및 house/env 발행

node_b
조명·창문 액추에이터 실행 및 상태 발행

node_c
판단 로직 수행, 모드 관리, 명령 발행

web_console
실시간 상태 표시 및 수동 제어 UI

MQTT Broker
모든 노드 간 메시지 중계 허브

팀원 역할 분배

정귀인

node_c · 웹 콘솔 · 통합

- 중앙 제어 로직 구현
- MQTT 연동 및 통신 구조 설계
- Flask 웹 콘솔 개발
- 통합 테스트 정리

황지용

node_b · 하드웨어

- 액추에이터 노드 구현
- WS2812 및 서보 제어
- 하드웨어 배선
- 모형 제작

김도경

node_a · 문서작업

- 센서 노드 작업
- 센서 실습 및 연동 지원
- PPT 초안 작성
- 모형 제작 지원

협업 구조 요약

→ 각자 독립 노드를 담당하여 개발 후 MQTT 브로커를 통해 통합

→ 개별 TEST → 부분 통합 → 전체 통합 테스트 순서로 진행

→ 역할은 분리되어 있으나 인터페이스(MQTT 토픽)는 공동 설계

하드웨어 및 개발 환경

하드웨어 구성

구분	부품	수량	용도
MCU	Raspberry Pi Pico 2 W	3	node_a / node_b / node_c
센서	CDS 조도 센서	1	조도 측정 (GP27)
센서	DHT 온습도 센서	1	온도·습도 측정 (GP15)
액추에이터	WS2812 RGB Strip	1	조명 제어 (GP16)
액추에이터	SG90 서보모터	1	창문 제어 (GP14)
기타	브레드보드, 점퍼선	—	배선 연결

소프트웨어 / 개발 환경

구분	기술 스택
MCU 펌웨어	C + Pico SDK
웹 서버	Python + Flask
통신 프로토콜	Wi-Fi + MQTT
MQTT Broker	Mosquitto (로컬 또는 외부)

주요 핀 구성 요약

노드	핀	연결 부품
node_a	GP15	DHT 온습도 센서
node_a	GP27	CDS 조도 센서
node_b	GP14	SG90 서보 (창문)
node_b	GP16	WS2812 RGB Strip

MQTT 통신

MQTT 주요 토픽

토픽 (Topic)	발행 (Pub)	구독 (Sub)	의미
house/env	node_a	node_c	환경 데이터
house/mode	web	node_c	모드 전환
house/cmd/light	node_c web	node_b	조명 제어
house/cmd/window	node_c web	node_b	창문 제어
house/status/nodeB	node_b	node_c web	액추에이터 상태
house/status/nodeC	node_c	web	제어기 상태
house/heartbeat/nodeA	node_a	node_c	node_a 생존
house/heartbeat/nodeB	node_b	node_c	node_b 생존
house/heartbeat/nodeC	node_c	web	node_c 생존

토픽 그룹

- **ENV** 환경 데이터 수집 채널
- **CMD** 장치 제어 명령 채널
- **STATUS** 상태 동기화 채널
- **HEARTBE** 노드 생존 확인 채널

통신 흐름 요약

node_a → (house/env) → **node_c** → (house/cmd/*) → node_b

web_console

상태 및 모니터링:
node_b와 node_c는 house/status/* 및 house/heartbeat/* 토픽을 통해 web_console로 상태를 지속적으로 발행합니다.

임계값 기반 자동 판단 — 사람 개입 없이 환경 제어

1. 센서 측정 및 발행

node_a: 조도·온도·습도 측정
→ house/env 발행

2. AUTO 모드 확인

node_c: 수신 후 현재 모드 확인
(MANUAL이면 명령 무시)

3. 임계값 판단 및 명령 발행

node_c: 센서값 기준 판단
→ house/cmd/light , window 발행

4. 장치 제어

node_b: WS2812(조명), 서보(창문) 제어

5. 상태 갱신

node_b/c: 상태 발행
→ web_console UI 갱신

자동 제어 기준표

센서	조건	동작
조도 (light)	< 280	조명 ON
조도 (light)	> 320	조명 OFF
온도 (temp)	> 28°C	창문 OPEN
습도 (humidity)	> 70%	창문 OPEN
온도 + 습도	≤ 27°C & ≤ 65%	창문 CLOSE

핵심 로직

- **중앙 집중형 판단:** 센서 노드(node_a)는 측정만 수행하고, 제어기(node_c)가 모든 판단을 담당하여 로직을 한 곳에서 관리합니다.
- **히스테리시스 적용:** 조명 ON(<280)과 OFF(>320) 기준을 다르게 설정하여 경계값 부근에서 장치가 깜빡이는 현상(Chattering)을 방지합니다.

사용자 직접 제어 — AUTO 모드에서는 수동 명령 차단

1. 모드 전환
사용자가 web_console에서
Mode **MANUAL** 선택

2. MANUAL 모드 진입
house/mode = MANUAL 발행
→ node_c 수동 명령 허용 상태

3. 수동 명령 입력
사용자가 Light 또는 Window 버튼 선택
→ house/cmd/* 발행

4. 장치 제어
node_b가 실제 장치 제어
(WS2812, 서보모터)

5. 상태 반영 및 갱신
node_b / node_c 상태 발행
→ web_console 상태 갱신

핵심 설계 포인트

AUTO 모드에서는 수동 명령이 무시됩니다.
node_c가 현재 모드를 확인하고, AUTO 상태일 때 web_console에서 들어오는 cmd 명령을 원천 차단하여 정책 일관성을 유지합니다.

모드 비교표

항목	AUTO 모드	MANUAL 모드
제어 주체	node_c (임계값 기반)	사용자 (web_console)
수동 명령	✕무시 (차단)	✓허용
센서 데이터	계속 수신	계속 수신
상태 발행	유지	유지

웹 콘솔

웹 콘솔 주요 기능

Environment
조도·온도·습도 실시간 센서 데이터 표시

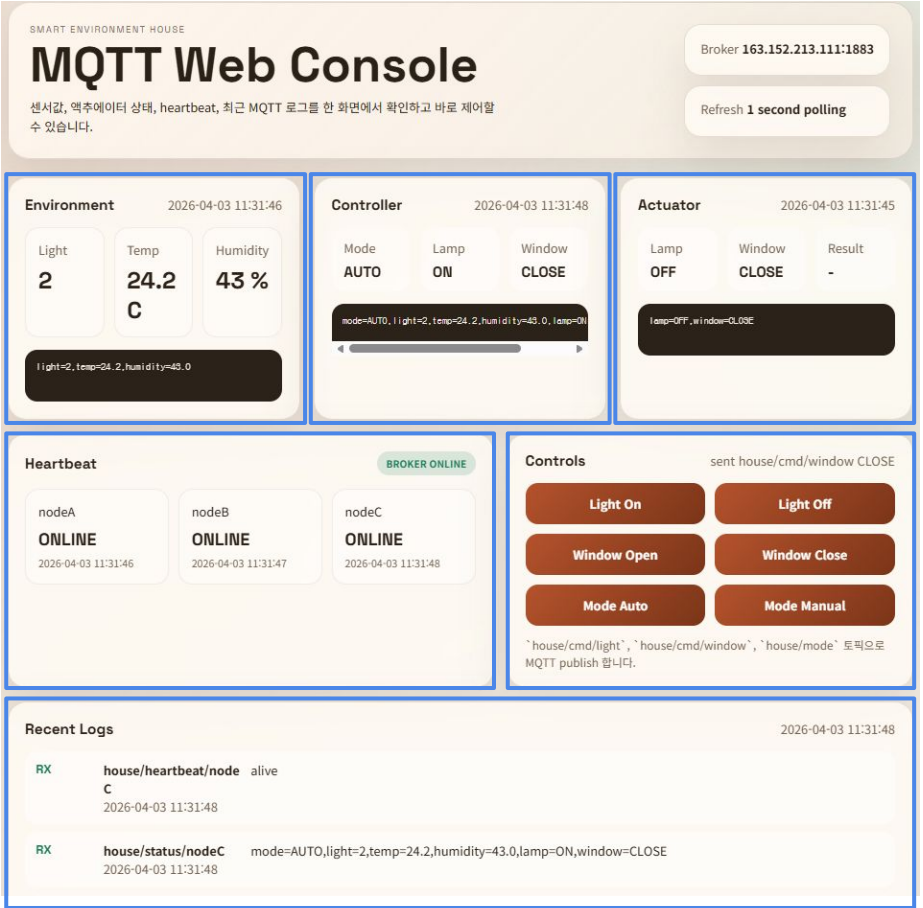
Controller (node_c)
현재 모드(AUTO/MANUAL) 및 판단 상태 표시

Actuator (node_b)
조명 및 창문 현재 동작 상태 표시

Heartbeat
각 노드(A, B, C) 생존 상태 및 연결 확인

MQTT 로그
최근 송수신된 MQTT 메시지 목록 표시

수동 제어 버튼
Light ON/OFF, Window OPEN/CLOSE 명령 전송



핵심코드 리뷰

핵심 제어 로직

node_c 임계값 기반 자동 제어 (node_c_controller.c)

```
if (controller->env.light < controller->thresholds.light_on_threshold) {
    set_lamp_state(controller, NODE_C_SWITCH_ON, "light below threshold");
} else if (controller->env.light > controller->thresholds.light_off_threshold) {
    set_lamp_state(controller, NODE_C_SWITCH_OFF, "light above threshold");
}

if (controller->env.temp > controller->thresholds.window_open_temp_threshold ||
    controller->env.humidity > controller->thresholds.window_open_humidity_threshold) {
    set_window_state(controller, NODE_C_WINDOW_OPEN, "temp or humidity high");
}
```

node_c 는 환경 값을 파싱한 뒤 조명과 창문 제어를 각각 판단.

핵심코드 리뷰

핵심 제어 로직

AUTO 모드에서 수동 명령 차단 (node_c_controller.c)

```
static bool ensure_manual_mode(node_c_controller_t *controller, const char *source, const char *payload)
{
    if (controller->mode == NODE_C_MODE_MANUAL) {
        return true;
    }

    node_c_logf(controller, "[NODE_C] WARN manual command ignored in AUTO mode from %s: %s", source, payl
    return false;
}
```

auto 모드 일때 웹이나 **MQTT**에서 들어온 수동 명령을 무시.

⇒ 사용자가 실수로 버튼을 눌러도 자동제어 정책이 깨지지 않음.

핵심코드 리뷰

핵심 제어 로직

중복 명령 방지 (node_c_controller.c)

```
static void set_lamp_state(node_c_controller_t *controller, node_c_switch_state_t next_state, const char
{
    if (controller->lamp_state == next_state) {
        return;
    }

    controller->lamp_state = next_state;
    node_c_publish(controller, NODE_C_TOPIC_CMD_LIGHT, lamp_name(next_state));
}
```

현재 장치 상태와 같은 명령은 다시 발행하지 않도록 설계.

⇒ 불필요한 **MQTT** 트래픽과 액추에이터 반복 동작을 줄이도록 함.

핵심코드 리뷰

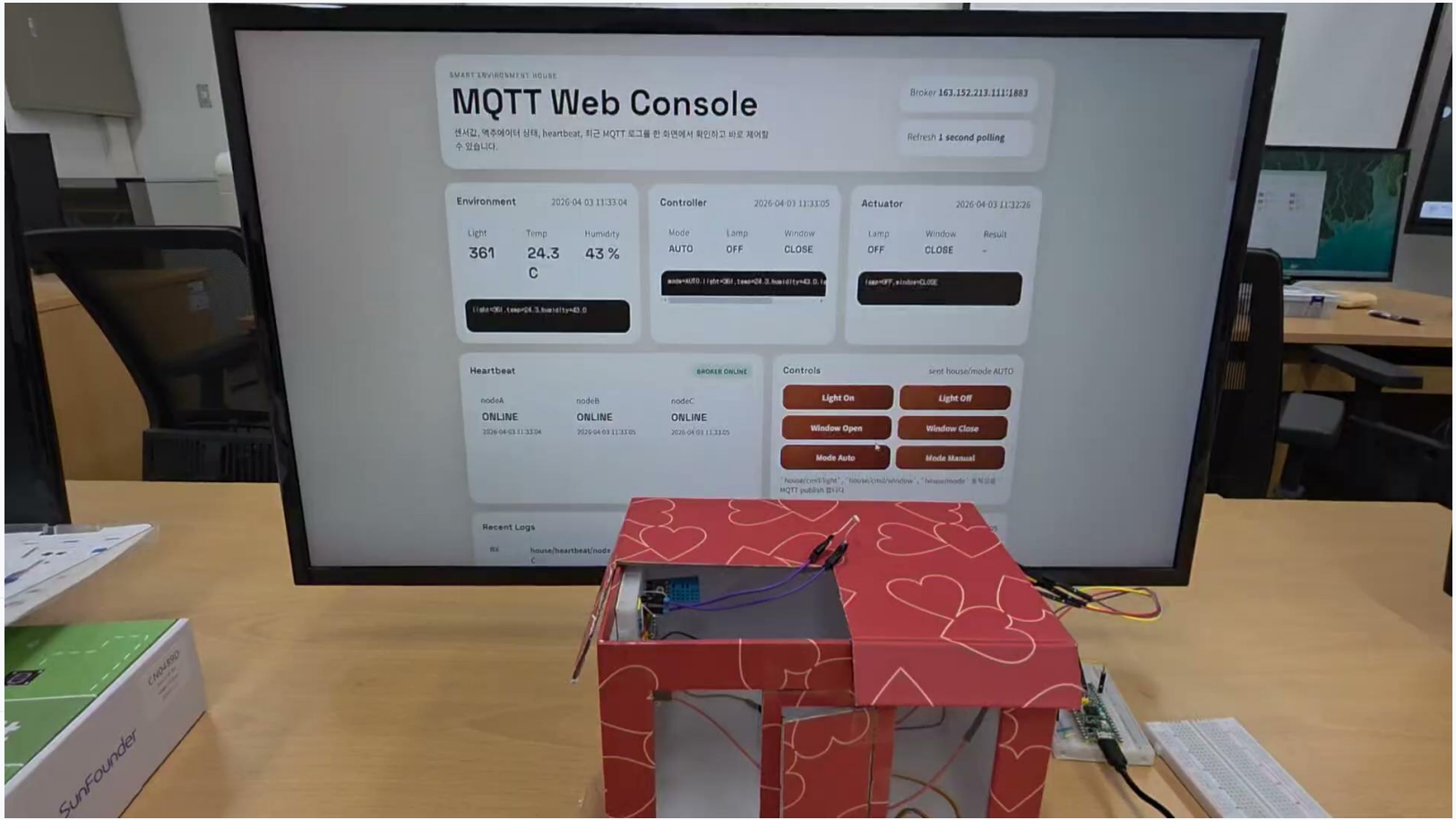
핵심 제어 로직

노드 상태 동기화

```
snprintf(payload, sizeof(payload), "mode=%s,light=%d,temp=%.1f,humidity=%.1f,lamp=%s,window=%s",
    mode_name(controller->mode),
    controller->env.light,
    controller->env.temp,
    controller->env.humidity,
    lamp_name(controller->lamp_state),
    window_name(controller->window_state));
```

node_b는 실제 반영 결과를 **house/status/nodeB**로 다시 알려주고 , **node_c**는 이를 반영해 전체 상태를 정리.
제어 명령만 보내는 구조가 아니라 결과확인까지 고려함.

시연 영상


SMART ENVIRONMENT HOUSE
MQTT Web Console
센서값, 액추에이터 상태, heartbeat, 최근 MQTT 로그를 한 화면에서 확인하고 바로 제어할 수 있습니다.
Broker 163.152.213.111:1883
Refresh 1 second polling
Environment
2026-04-03 11:33:04
Light
361
Temp
24.3 C
Humidity
43 %
Controller
2026-04-03 11:33:05
Mode
AUTO
Lamp
OFF
Window
CLOSE
Actuator
2026-04-03 11:32:26
Lamp
OFF
Window
CLOSE
Result
-
Heartbeat
BROKER ONLINE
nodeA
ONLINE
nodeB
ONLINE
nodeC
ONLINE
Controls
sent house/mode AUTO
Light On
Light Off
Window Open
Window Close
Mode Auto
Mode Manual
Recent Logs
SunFounder

테스트 결과

통합 테스트 결과

- node_a + node_c + node_b + web_console 전체 MQTT 흐름 확인
- node_b + node_c + web_console 부분 통합 테스트 확인
- web_console + node_c만으로 상태 확인 및 수동 제어 가능
- node_c smoke test 통과
- heartbeat 수신 및 timeout 경고 동작 확인
- status snapshot 발행 및 상태 동기화 확인
- 중복 명령 방지 로직 동작 확인

동작 흐름 요약

시나리오	입력	결과
자동 제어	센서값 임계값 초과	조명/창문 자동 제어
수동 제어	웹 버튼 클릭 (MANUAL)	장치 즉시 반응
상태 확인	웹 콘솔 접속	실시간 상태 표시
노드 감시	heartbeat 수신	생존 상태 확인

프로젝트 강점

역할 분리

센서·판단·실행·UI가 독립 노드로 분리된 명확한 구조로 시스템 복잡도를 낮춤

MQTT 확장성

토픽 추가만으로 새로운 센서 노드나 액추에이터 기능을 쉽게 확장 가능

운영 가시성

heartbeat와 status snapshot을 통해 전체 시스템의 실시간 운영 상태 확인 가능

실제 동작성

WS2812 8픽셀 조명과 서보 창문을 통해 실제 스마트 홈IoT의 구동을 구현

프로젝트 진행 중 어려웠던 점 및 해결 방안

협업 및 통합 문제

- × GitHub 활용 중 실시간 코드 충돌 발생 가능성
- × 첫 협업 프로젝트로 인한 통합 과정의 시행착오

해결 방안

- → **MCU 역할 명확히 분리:** 독립적 개발 및 테스트 후 최종 통합
- → **문서화:** 구현 기능과 통합 주의사항을 텍스트로 정리하여 기준 통일

창문 구동 구조 문제

- × 창문의 회전축과 서보모터의 회전축 불일치
- × 구조적 한계로 인한 자동 개폐 구현의 어려움

해결 방안

- → **구조 검토 및 재설계:** 추가 조사와 구조 검토를 통해 싱글 서보 기반 창문 구동 구조를 설계
- → **실제 구현:** 재설계된 구조를 통해 안정적인 자동 개폐 기능 완성

결론 및 향후 계획

한계점 (Limitations)

배선 및 전원 안정성 이슈 가능

브레드보드 기반 연결로 인한 물리적 불안정성

센서와 액추에이터 배치에 따른 측정/제어 간섭 가능

조도센서가 실내 LED의 영향을 받을 수 있어 배치 최적화

환경별 임계값 보정 미적용

현재 고정된 임계값을 사용 중이며, 동적 보정 필요

향후 계획

계획	내용
데이터 저장	센서 이력 DB 저장 및 대시보드 시각화
디스플레이	현장 노드에 OLED/LCD 상태 표시 추가
알림 기능	이상 상태 발생 시 알림
하드웨어 확장	가스 센서, 환풍기 등 추가 센서 및 액추에이터 연결

QnA